Машинное обучение

Лекция 5: Нейронные сети

Дьулустан Никифоров

Кафедра ИТ
Северо-Восточный Федеральный Университет

Весна 2026

Введение

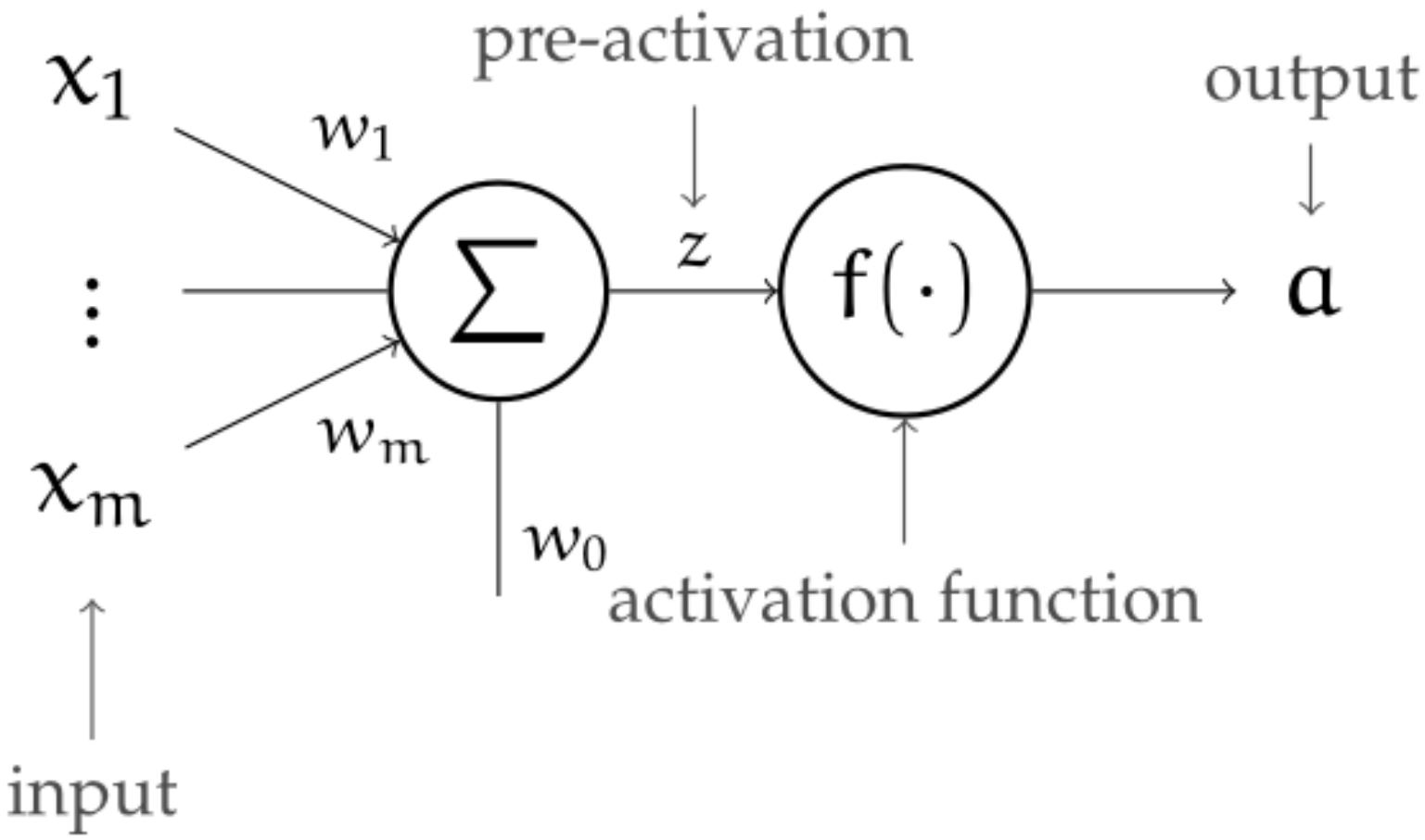
- Мы теперь переходим к теме с самым хайповым названием — нейронные сети.
- Как и идея персептрона, впервые модель нейросетей появилась уже давно, в 1943 году.

На нейросети можно посмотреть с разных точек зрения:

1. Модель, вдохновленная строением человеческого мозга — мозг тоже просто большая сеть нейронов!
2. Применение стохастического градиентного спуска для задач классификации и регрессии, с *очень* большим классом гипотез.
3. Способ разработки приложений, которые делают предсказания на основе огромных массивов данных.

Нейрон

Самый базовый элемент нейросетей — это **нейрон (neuron)** или **узел (node)**.



Нейрон

$x \in \mathbb{R}^m$ — вектор инпута;
$a \in \mathbb{R}$ — значение аутпута;
$(w_1, ..., w_m) \in \mathbb{R}^m$ — вектор **весов** — это параметры модели;
w_0 — смещение / **офсет** (offset) — тоже параметр;
$f : \mathbb{R} \to \mathbb{R}$ — **функция активации**;

$$a = f(z) = f\left(\sum_{j=1}^{m} x_j w_j + w_0\right) = f(w^T x + w_0)$$

Просто один нейрон — уже является моделью, которая может решать задачу классификации или регрессии.

Для датасета $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$, можно выбрать функцию потерь $L(g, a)$ и решать оптимизационную задачу:

$$J(w, w_0) = \sum_{i=1}^{n} L(NN(x^{(i)}; w, w_0), y^{(i)}) \to \min_{w, w_0}$$

Оптимизация нейрона

Для датасета $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$, можно выбрать функцию потерь $L(g, a)$ и решать оптимизационную задачу:

$$J(w, w_0) = \sum_{i=1}^{n} L\left(NN(x^{(i)}; w, w_0), y^{(i)}\right) \to \min_{w, w_0}$$

$NN(x^{(i)}; w, w_0)$ — аутпут нейрона (нейронной сети) для точки $x^{(i)}$.

Мы уже решали эту задачу пару раз. Помните?

Оптимизация нейрона

Логистическая регрессия, использующая функцию потерь L_{nll} и функцию активации $f(x) = \sigma(x)$:

$$J_{lr}(\theta, \theta_0) = \left(\frac{1}{n}\sum_{i=1}^{n} L_{nll}((\theta^T x^{(i)} + \theta_0), y^{(i)})\right) + \lambda\|\theta\|^2 \to \min_{\theta,\theta_0}$$

Линейная регрессия, использующая квадратичную функцию потерь и функцию активации $f(x) = x$:

$$J(\theta, \theta_0) = \frac{1}{n}\sum_{i=1}^{n} \left(\theta^T x^{(i)} + \theta_0 - y^{(i)}\right)^2 \to \min_{\theta,\theta_0}$$

Нейронная сеть

Несколько нейронов объединяются в сеть (network).

Сеть принимает инпут $x \in \mathbb{R}^m$ и выводит $a \in \mathbb{R}^n$.

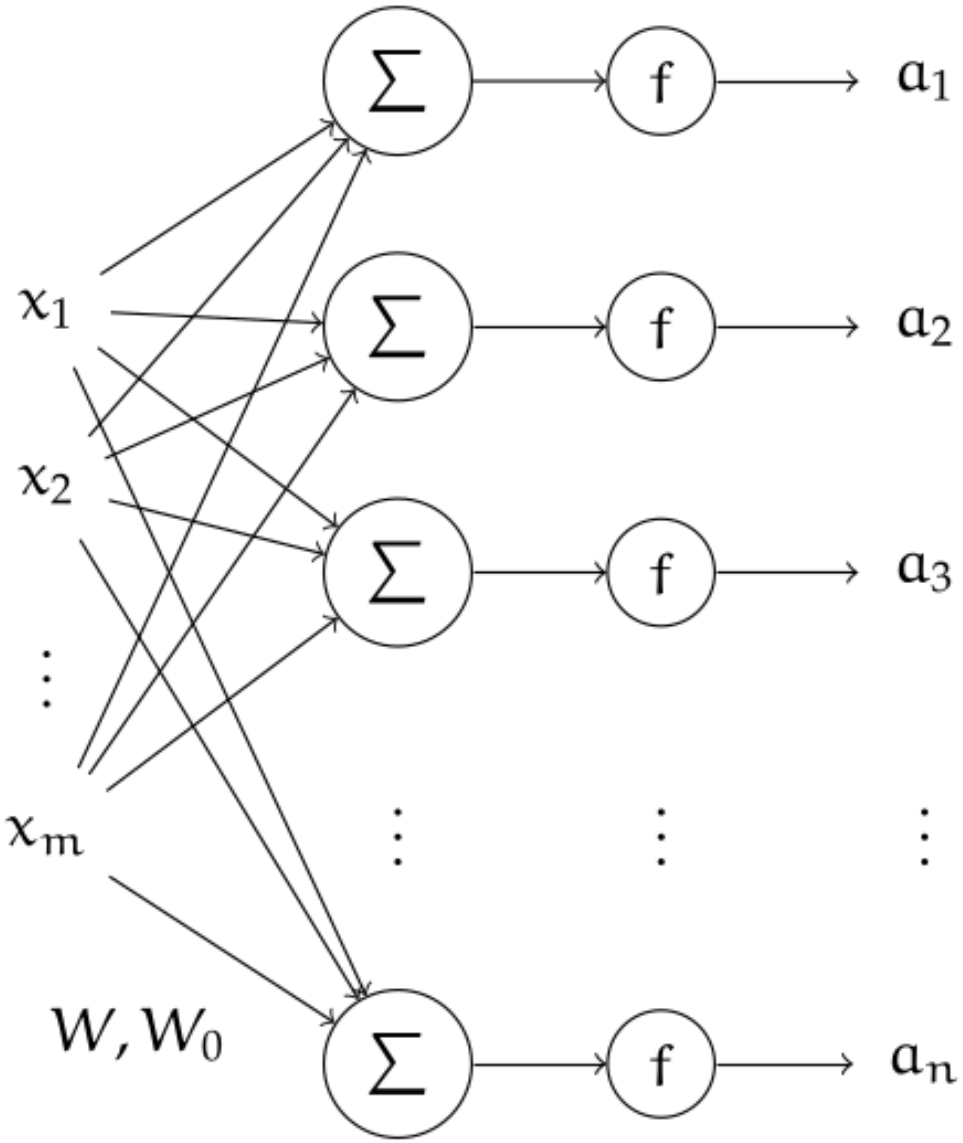
Каждый нейрон может принимать свой инпут из x и аутпутов других нейронов.

Сегодня рассматриваем только сети типа **feed-forward** — инпут каждого нейрона *не зависит* от его же аутпутов.
Иными словами, сеть является *ациклическим ориентированным графом*.

Такую ациклическую сеть можно представить как набор последовательных **слоёв (layers)**:

- Внутри одного слоя нету связей между нейронами.
- Связи идут только между соседними слоями.

Нейронный слой



Нейронный слой

- $X = (x_1, ..., x_m)^T \in \mathbb{R}^m$ — инпут слоя;
- $A = (a_1, ..., a_n)^T \in \mathbb{R}^n$ — аутпут слоя — называют **активацией** слоя;
- W — матрица $m \times n$ весов слоя;
- W_0 — вектор $n \times 1$ офсетов слоя.
- $Z = W^T X + W_0$ — вектор $n \times 1$ — называют **пре-активацией** слоя.

$$A = f(Z) = f(W^T X + W_0)$$

Функция активации применяется к пре-активационным значениям Z.

Смысл нейронных слоёв

На что очень похоже $A = f(W^T X + W_0)$?

Смысл нейронных слоёв

На что очень похоже $A = f(W^T X + W_0)$?

- На линейный классификатор:
  $h(x; \theta, \theta_0) = sign(\theta^T x + \theta_0)$
- На линейную регрессию:
  $h(x; \theta, \theta_0) = \theta^T x + \theta_0$

Смысл нейронных слоёв

- Один нейронный слой позволяет работать только с **линейными** гипотезами.
- Добавление других слоёв позволяет нам работать с **нелинейными** гипотезами.

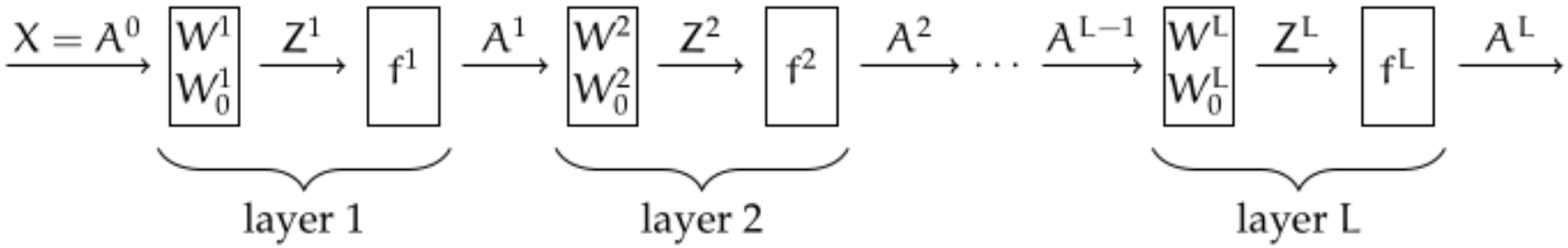
Мы знаем, что линейный классификатор / регрессия могут добиться нелинейности с помощью нелинейной трансформации признаков $x \to \phi(x)$ (*представление признаков*).

С такой перспективы, нейронная сеть — это тоже линейный классификатор / регрессия (последний слой) с нелинейной трансформацией (все предыдущие слои неявно делают то же самое).

Много нейронных слоёв

Введём такие обозначения:

- l — номер нейронного слоя;
- m^l — количество инпутов, входящих в слой l;
- n^l — количество аутпутов, выходящих из слоя l;
- W^l — матрица $m^l \times n^l$ весов слоя l;
- W_0^l — матрица $n^l \times 1$ офсетов слоя l;
- $Z^l = W^{l^T} A^{l-1} + W_0^l$ — преактивация слоя l;
- $A^l = f^l(Z^l)$ — активация (аутпут) слоя l.



Много нейронных слоёв

Обратим внимание, что значения, идущие между соседними слоями всегда проходят через функции активации f^l.

А что, если мы не будем использовать функции активации, т.е. возьмем $f(x) = x$?

Много нейронных слоёв

Обратим внимание, что значения, идущие между соседними слоями всегда проходят через функции активации f^l.

А что, если мы не будем использовать функции активации, т.е. возьмем $f(x) = x$?

$$
\begin{aligned}
A^L &= W^{L^T} A^{L-1} + W_0^L = W^{L^T}(W^{L-1^T} A^{L-2} + W_0^{L-1}) + W_0^L \\
&= ... \\
&= W^{L^T} W^{L-1^T} ... W^{1^T} X + ... \\
&= \tilde{W}^L X + \tilde{W}_0
\end{aligned}
$$

Тогда вся сеть сводится к одному слою!

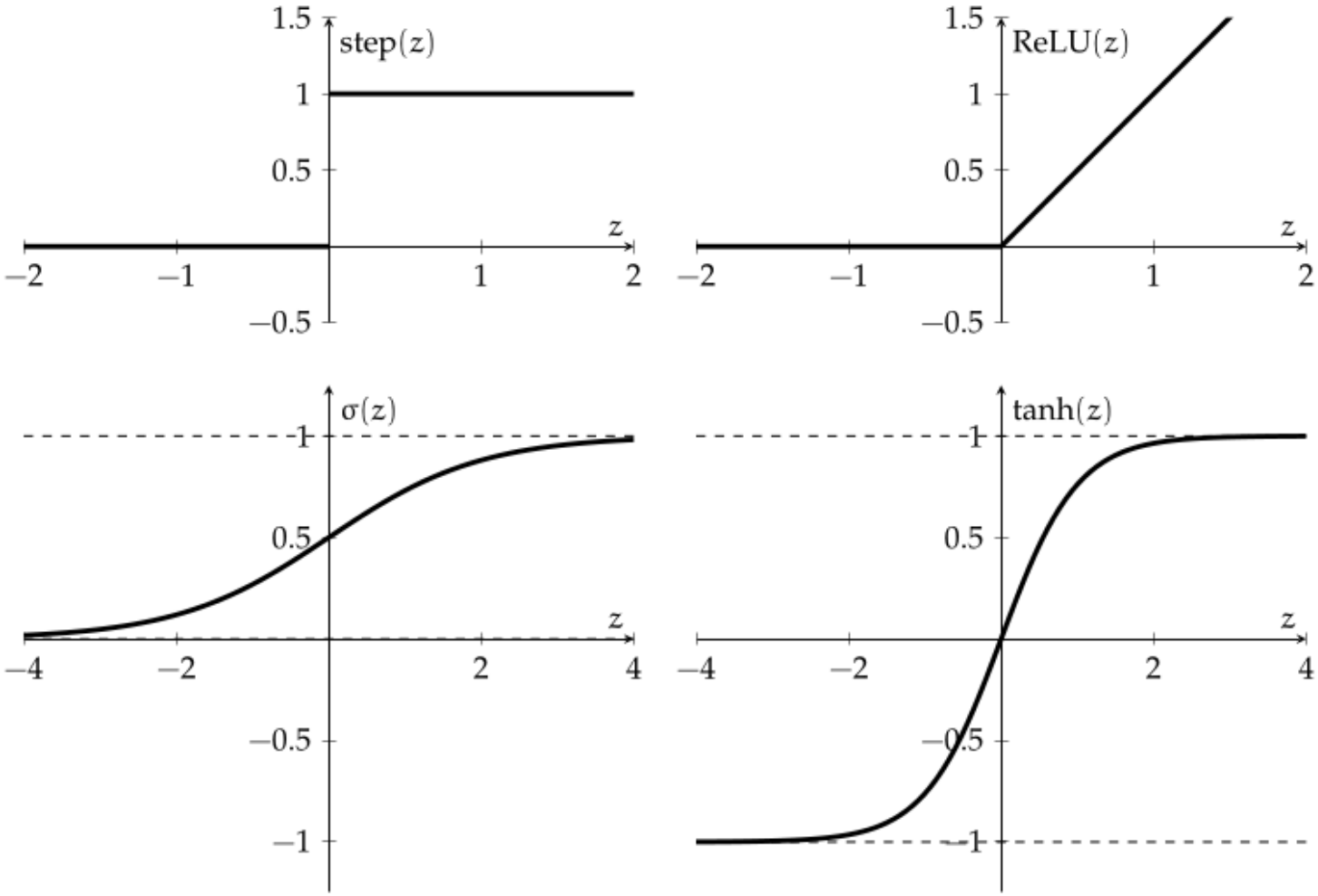
Функция активации

Значит, функция активации имеет важное значение в структуре нейросети.

Рассмотрим некоторые популярные функции активации:

- **Step function:**
  $$step(z) = \begin{cases} 0 \text{ если } z < 0 \\ 1 \text{ иначе} \end{cases}$$
- **Rectified linear unit:**
  $$ReLU(z) = \begin{cases} 0 \text{ если } z < 0 \\ 1 \text{ иначе} \end{cases} = \max(0, z)$$
- **Sigmoid function (logistic function):**
  $\sigma(z) = \frac{1}{1+e^{-z}}$
- **Hyperbolic tangent:**
  $tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Функция активации



Функция активации

- В чем проблема со step-функцией?

Функция активации

- В чем проблема со step-функцией? Градиентный спуск вообще не работает с ней.
- Сигмоид-функция — «разглаженная» версия step-функции с хорошей непрерывной производной.

Ещё одна популярная функция активации:

- **Softmax function:** Берёт $Z \in \mathbb{R}^n$ и возвращает $A \in [0,1]^n$ такую, что $\sum_{i=1}^n A_i = 1$:

$$softmax(z) = \begin{pmatrix} \exp(z_1)/\sum_i \exp(z_i) \\ \vdots \\ \exp(z_n)/\sum_i \exp(z_i) \end{pmatrix}$$

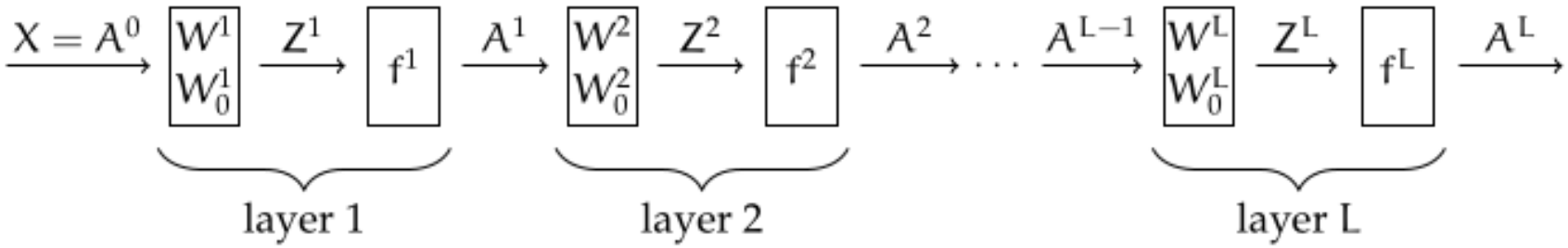
Дифференцирование нейросети

Теперь мы поняли, как устроена нейронная сеть. Осталось научиться обучать её на датасетах.
Мы хотим градиентным спуском минимизировать её потери на датасете:

$$Loss(NN(x;W),y) = \sum_{i=1}^{n} Loss(NN(x^{(i)};W),y^{(i)}) \to \min_{W},$$

где W представляет собой все веса W^l, W_0^l всех слоёв $(l = 1, ..., L)$.
Для этого нам посчитать $\nabla_W Loss(NN(x;W),y)$.
Будем считать градиент для конкретной точки (x, y).

Дифференцирование нейросети



$$Loss(NN(x;W),y) = Loss(A^L, y)$$
$$A^L = f^L(Z^L),\ Z^L = W^{L^T} A^{L-1}$$

$$\frac{\partial Loss}{\partial W^L} = \frac{\partial Loss}{\partial A^L} \cdot \underbrace{\frac{\partial A^L}{\partial Z^L}}_{(f^L)'} \cdot \underbrace{\frac{\partial Z^L}{\partial W^L}}_{A^{L-1}}$$

$$\underbrace{\frac{\partial Loss}{\partial W^l}}_{m^l \times n^l} = \underbrace{A^{l-1}}_{m^l \times 1} \underbrace{\left(\frac{\partial Loss}{\partial Z^l}\right)^T}_{1 \times n^l}, \ l = 1, ..., L$$

Дифференцирование нейросети

Следуя цепному правилу дифференцирования, мы можем вывести производную по Z^1:

$$\frac{\partial Loss}{\partial Z^1} = \frac{\partial Loss}{\partial A^L} \cdot \frac{\partial A^L}{\partial Z^L} \cdot \frac{\partial Z^L}{\partial A^{L-1}} \cdot \frac{\partial A^{L-1}}{\partial Z^{L-1}} \cdot \ldots \cdot \frac{\partial A^2}{\partial Z^2} \cdot \frac{\partial Z^2}{\partial A^1} \cdot \frac{\partial A^1}{\partial Z^1}$$

- $\frac{\partial Loss}{\partial A^L} \to n^L \times 1$, зависит от выбранной функции потерь;
- $\frac{\partial Z^l}{\partial A^{l-1}} = W^l \to m^l \times n^l$;
- $\frac{\partial A^l}{\partial Z^l} \to n^l \times n^l$; все недиагональные элементы равны нулю и на диагоналях стоят $\frac{\partial a_i^l}{\partial z_i^l} = f^{l\prime}(z_i^l)$

В итоге,

$$\frac{\partial Loss}{\partial Z^l} = \frac{\partial A^l}{\partial Z^l} \cdot W^{l+1} \cdot \frac{\partial A^{l+1}}{\partial Z^{l+1}} \cdot \ldots \cdot W^{L-1} \cdot \frac{\partial A^{L-1}}{\partial Z^{L-1}} \cdot W^L \cdot \frac{\partial A^L}{\partial Z^L} \cdot \frac{\partial Loss}{A^L}$$

Error back-propagation

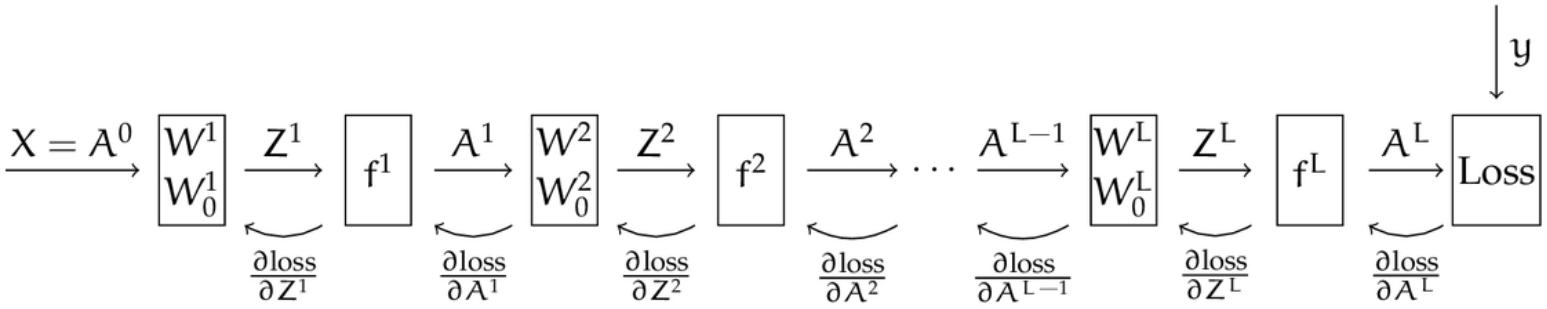
$$\frac{\partial Loss}{\partial W^l} = A^{l-1}\left(\frac{\partial Loss}{\partial Z^l}\right)^T$$

$$\frac{\partial Loss}{\partial Z^l} = \frac{\partial A^l}{\partial Z^l} \cdot W^{l+1} \cdot \frac{\partial A^{l+1}}{\partial Z^{l+1}} \cdot \ldots \cdot W^{L-1} \cdot \frac{\partial A^{L-1}}{\partial Z^{L-1}} \cdot W^L \cdot \frac{\partial A^L}{\partial Z^L} \cdot \frac{\partial Loss}{A^L}$$

Это позволяет нам найти градиент потерь нейросети относительно любых W^l.

$$\begin{tikzcd}
 & & & & & & & & & & & & & & y \arrow[d] \\
X = A^0 \arrow[r] & \boxed{\begin{matrix} W^1 \\ W_0^1 \end{matrix}} \arrow[r, bend left, "Z^1"] & \boxed{f^1} \arrow[l, bend left, "\frac{\partial loss}{\partial Z^1}"] \arrow[r, bend left, "A^1"] & \boxed{\begin{matrix} W^2 \\ W_0^2 \end{matrix}} \arrow[l, bend left, "\frac{\partial loss}{\partial A^1}"] \arrow[r, bend left, "Z^2"] & \boxed{f^2} \arrow[l, bend left, "\frac{\partial loss}{\partial Z^2}"] \arrow[r, bend left, "A^2"] & \cdots \arrow[l, bend left, "\frac{\partial loss}{\partial A^2}"] \arrow[r, bend left, "A^{L-1}"] & \boxed{\begin{matrix} W^L \\ W_0^L \end{matrix}} \arrow[l, bend left, "\frac{\partial loss}{\partial A^{L-1}}"] \arrow[r, bend left, "Z^L"] & \boxed{f^L} \arrow[l, bend left, "\frac{\partial loss}{\partial Z^L}"] \arrow[r, bend left, "A^L"] & \boxed{Loss} \arrow[l, bend left, "\frac{\partial loss}{\partial A^L}"]
\end{tikzcd}$$

Error back-propagation



Процесс выглядит так:

- сначала **forward pass** — начиная с первого слоя, вычисляются все A и Z, и в конце, сам Loss нейросети на точке (датасете);
- потом **error back-propagation** — начиная с последнего слоя L, идём обратно и вычисляем градиенты $\frac{\partial Loss}{\partial W^L}$.

С помощью этого алгоритма, мы можем вычислять градиенты целевой функции нейросети относительно всех весов $\Rightarrow$ мы можем обновлять (корректировать) их значения — т.е. делать градиентный спуск.

Инициализация W

- Для нейросети её целевая функция — это супермногоразмерная функция очень сложного вида $\Rightarrow$ вообще не идёт речи о никакой выпуклости.
- Следовательно, у неё может быть много локальных минимумов, в которых градиентны спуск может застрять.
- Ещё хуже, что типичные функции активации имеют нулевые (околонулевые) производные при больших значениях аргументов $\Rightarrow$ неудачный исходный выбор весов может привести к провалу.
- Например, если просто возьмем нулевые веса в начале $\Rightarrow$ спуск вообще не начнётся.
- Поэтому инициализация исходных значений W крайне важно!

Инициализация W

- Все эти проблемы можно решить тем, что мы будем *случайно и независимо* генерировать все веса.
- При этом хотим чтобы значения весов не были слишком большими по модулю.

Хороший выбор: генерировать по нормальному распределению.

$$W_{ij}^{l} \sim Gaussian\left(0, \frac{1}{m^{l}}\right)$$

здесь m^{l} — количество инпутов в слое l.

Стохастический градиентный спуск на нейросети

У нас есть нейросеть с L слоями, в слой l заходят m_l инпутов и используется функция активации f_l.
Мы обучаем нейросеть на датасете $\mathcal{D}_n$ из n точек с помощью **стохастического градиентного спуска** с T итерациями:

SGD-NEURAL-NET$(\mathcal{D}_n, T, L, (m^1, \ldots, m^L), (f^1, \ldots, f^L))$

```
for l = 1 to L
    W^l_ij ~ Gaussian(0, 1/m^l)
    W^l_0j ~ Gaussian(0, 1)
for t = 1 to T
    i = random sample from {1, ..., n}
    A^0 = x^(i)
    // forward pass to compute the output A^L
    for l = 1 to L
        Z^l = W^lT A^(l-1) + W^l_0
        A^l = f^l(Z^l)
```

Стохастический градиентный спуск на нейросети

```
loss = Loss(A^L, y^(i))
for l = L to 1:
    // error back-propagation
    ∂loss/∂A^l = if l < L then ∂loss/∂Z^(l+1) · ∂Z^(l+1)/∂A^l else ∂loss/∂A^L
    ∂loss/∂Z^l = ∂loss/∂A^l · ∂A^l/∂Z^l
    // compute gradient with respect to weights
    ∂loss/∂W^l = ∂loss/∂Z^l · ∂Z^l/∂W^l
    ∂loss/∂W_0^l = ∂loss/∂Z^l · ∂Z^l/∂W_0^l
    // stochastic gradient descent update
    W^l = W^l − η(t) · ∂loss/∂W^l
    W_0^l = W_0^l − η(t) · ∂loss/∂W_0^l
```

Часто $\frac{\partial loss}{\partial Z^l}$ легче вычислить напрямую, чем через вычисление $\frac{\partial loss}{\partial A^l}$ и $\frac{\partial A^l}{\partial Z^l}$ $\leftarrow$ в зависимости от выбранных функций активации и потери.

Negative log likelihood (NLL)

Пусть нейросеть решает задачу бинарной классификации: все точки имеют лейблы $y \in \{0, 1\}$.

Пусть функция активации на выходном слое (последний слой $l = L$) — сигмоид и только один нейрон на этом слое, т.е. нейросеть выдает скаляр $a^L \in [0, 1]$ в качестве ответа.

Мы можем интерпретировать ответ нейросети $a^{(i)}$ для точки $(x^{(i)}, y^{(i)})$ как вероятность, которую нейросеть приписывает событию $y^{(i)} = 1$.

Соответственно, мы хотим максимизировать вот эту «вероятность», которую называют **likelihood**:

$$\prod_{i=1}^{n} \begin{cases} a^{(i)}, \text{ если } y^{(i)} = 1 \\ 1 - a^{(i)}, \text{ иначе} \end{cases}$$

Negative log likelihood (NLL)

Likelihood можно переписать так:

$$\prod_{i=1}^{n} a^{(i)^{y^{(i)}}} (1 - a^{(i)})^{1-y^{(i)}}$$

Максимизация Likelihood эквивалентно максимизации его логарифма, называемого **log likelihood**:

$$\sum_{i=1}^{n} y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log (1 - a^{(i)})$$

Осталось взять её со знаком минус, чтобы сделать задачу минизации потерь:

$$\sum_{i=1}^{n} \mathcal{L}_{nll}(a^{(i)}, y^{(i)}) = \sum_{i=1}^{n} (-(y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log (1 - a^{(i)})))$$

Negative log likelihood (NLL)

Так мы определил **negative log likelihood (NLL)** функцию потерь:

$$\mathcal{L}(g, a) = -\left(a \cdot \log g + (1 - a) \cdot \log(1 - g)\right)$$

NLL: много классов

Можно идею NLL расширить на многоклассовую классификацию!

Пусть лэйблы имеют вид $y = [y_1, y_2, ..., y_K]^T$, где только один из y_k равен единице: $y_k = 1$ означает, что точка принадлежит классу k. Это у нас классификация на K классов.

Пусть нейросеть использует softmax функцию активации на выходном слое $softmax(z) = \begin{pmatrix} \exp(z_1)/\sum_i \exp(z_i) \\ \vdots \\ \exp(z_n)/\sum_i \exp(z_i) \end{pmatrix}$.

Значит, нейросеть выдает ответы вида $a = [a_1, ..., a_K]^T$, где сумма $\sum_{k=1}^{K} = 1$ — можно интерпретировать как вероятностное распределение над K возможными классами.

NLL: много классов

$\prod_{k=1}^{K} a_k^{y^k}$ — likelihood;

$\sum_{k=1}^{K} y_k \log a_k$ — log likelihood;

$$\mathcal{L}_{nllm}(g, a) = -\sum_{k=1}^{K} a_k \cdot \log(g_k)$$

Это называется **Negative log likelihood multiclass (NLLM)**.

Сочетания функций активации и потери

Когда создается нейросеть, её функции активации (особенно на выходном слое) и функция потери отражают предположения о входных и выходных данных — в частности, их возможные диапазоны.

Функция активации выходного слоя и функция потери должны образовывать разумную комбинацию.

Вот популярные комбинации:

Loss	f^L
squared	linear
hinge	linear
NLL	sigmoid
NLLM	softmax